

NATIONAL ACADEMY OF PROFESSIONAL STUDIES

ITS203 – Object-Oriented Design and Programming

Assessment C – Milestone 1

Project Idea Proposal

Project Title: Personal Budget Tracker

Prepared by: Pratikshya Pokhrel

Student ID: S2400757

Submission Date: 28 August 2026

Word Count: 933

Milestone 1: Project Idea Proposal

Problem Description

For many people, notably students and young professionals, it can be hard to keep track of how they spend their money each month. Regular expenditure on things like food, transit and entertainment is typically paid for in little, fragmented sums, making it hard to understand the overall picture of your finances. This difficulty is consistent with mental accounting theory which describes how individuals naturally categorize income and expenditure into informal, loosely tracked mental accounts rather than a single coherent ledger, making it easy to lose sight of overall financial position unless an external tool reinforces the categorization (Thaler, 1999). This is not just a problem for those with comfortable incomes; Vines et al. (2014) found that structured tracking of income and outgoings is particularly helpful for those managing tight or irregular budgets, as it allows spending decisions to be made with a clearer view of what money is actually available. This software is for those who desire a simple, private way to keep track of their own income and expenses, without the complexity of an accounting tool or sharing data with a third party via an online banking app. It would aid them by providing a clear, categorized perspective of their expenditure and a straightforward monthly total, helping them to make more educated financial decisions.

Proposed Idea

The application that I propose is a Personal Budget Tracker developed as a C# Windows Forms desktop application. Users can add transactions (revenue or expense), assign category for each transaction (for example, food, travel, bills, entertainment or salary) and see a running list of all transactions. The application will calculate and display summary totals such as total income, total expenses and current balance, and will allow totals to be broken down by category so the user can see where most of their money is going. The program will be developed following object-oriented principles including encapsulation, inheritance and polymorphism as well as according to the design guidelines recommended for C# desktop applications (Microsoft, n.d.-b).

Inspiration

The idea started from seeing how easy it is to lose track of little routine expenses if they are not written down somewhere. If you are not comfortable with spreadsheets, a little app with specific buttons and instantaneous totals is much more approachable than a spreadsheet. This project is also a good fit for the unit as it naturally involves multiple related object types (transactions, categories, accounts) and lends itself to a real opportunity to use inheritance, encapsulation and exception handling in a way that is easy to explain and justify in the Milestone 3 oral presentation.

Functional Requirements

The program will support at least the following essential features:

- The user can add a new transaction by providing an amount, a date, a category, a description, and whether it is an income or an expense.
- In the main form, the user is able to see all the recorded transactions in a grid or list view.
- An existing transaction can be edited or deleted if it was wrongly entered.
- The program calculates and shows the total income, total expenses and current balance for the chosen time.
- The application adds up the total spent on each category (e.g. total spent on food this month) and shows this as a concise summary.
- Transactions can be filtered or viewed by category and by date period.
- The application checks user input (e.g. not allowing negative or non-numeric amount) and presents appropriate error message rather than crashing.

Non-Functional Requirements

- Users should not experience any lag (less than 1 second) in the application when performing operations such as creating or canceling a transaction.
- The interface should be clear and self-explanatory enough that a first time user should be able to add a transaction without any instructions.
- The application should gracefully handle invalid input using structured exception handling so that runtime problems are identified and reported without the program crashing or losing existing data (Microsoft, n.d.-a).

- Transactions input by the user should be stored between sessions, so they may be retrieved the next time the application is used.

Data Storage Plan

The program will be using file storage instead of a full database. This is because the data we are dealing with (list of transactions) is simple and a database would introduce unneeded complexity for a project of this scope. Transactions will be kept in a local JSON file (e.g. transactions.json) and will be serialized and de-serialised using the JSON support built into the .NET class library (Microsoft, n.d.-c). Each transaction will be stored as an object with these fields:

- Id (unique identification) Date Category (Food, Transport, Bills, Salary etc.)
- Description, Amount and Type (Income/Expense)

When the program starts up, it will read the JSON file (if present) and load any existing transactions into memory. Whenever a transaction is added, updated or deleted, the in-memory list will be re-saved so that changes are persisted between sessions. If time allows, this may be converted to a lightweight SQLite database later, but the JSON approach is good enough to meet all functional requirements for this project.

Development Plan

- Weeks 7-8: Create the Visual Studio solution and GitHub repository, and create the basic class structure (Transaction class and a management class to hold the transactions and total them).
- Weeks 8–9: Develop the Windows Forms interface to create, view, update and delete transactions and link it to the classes below.
- Week 10: Add category summaries, filtering, input validation and exception handling, ready for Milestone 2 progress report.
- Weeks 11-12: Improve the interface, do testing, write the README, and complete the reflection paper and the oral presentation.

References

- Microsoft. (n.d.-a). Exceptions and exception handling – C#. Microsoft Learn.
<https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/exceptions/>
- Microsoft. (n.d.-b). Object-oriented programming – C#. Microsoft Learn.
<https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/tutorials/oop>
- Microsoft. (n.d.-c). How to serialize and deserialize JSON using C# – .NET. Microsoft Learn.
<https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/how-to>
- Thaler, R. H. (1999). Mental accounting matters. *Journal of Behavioral Decision Making*, 12(3), 183–206. [https://doi.org/10.1002/\(SICI\)1099-0771\(199909\)12:3%3C183::AID-BDM318%3E3.0.CO;2-F](https://doi.org/10.1002/(SICI)1099-0771(199909)12:3%3C183::AID-BDM318%3E3.0.CO;2-F)
- Vines, J., Dunphy, P., & Monk, A. (2014). Pay or delay: The role of technology when managing a low income. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (pp. 501–510). Association for Computing Machinery.
<https://doi.org/10.1145/2556288.2556961>